



12.05.26



WEBSITE *TaskManager*

Collaborative Task Management System

PROFESSOR
**GULZADA
ESENALIEVA**

PRESENTED BY
ALI KHANTSZIAN



Table of Contents

1

PROJECT OVERVIEW

2

CORE FEATURES

3

TECHNICAL STACK

4

ARCHITECTURE OVERVIEW

5

DATA MODELS

6

SECURITY ARCHITECTURE

7

API ENDPOINTS

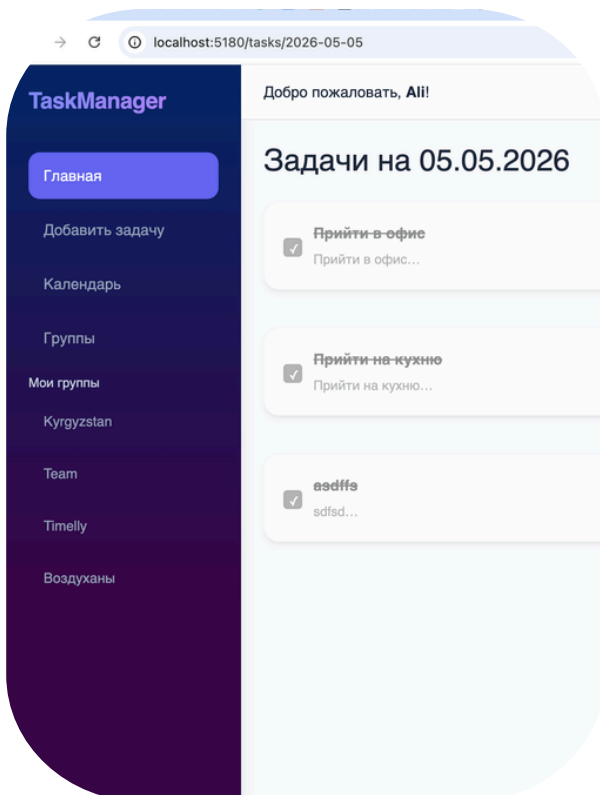
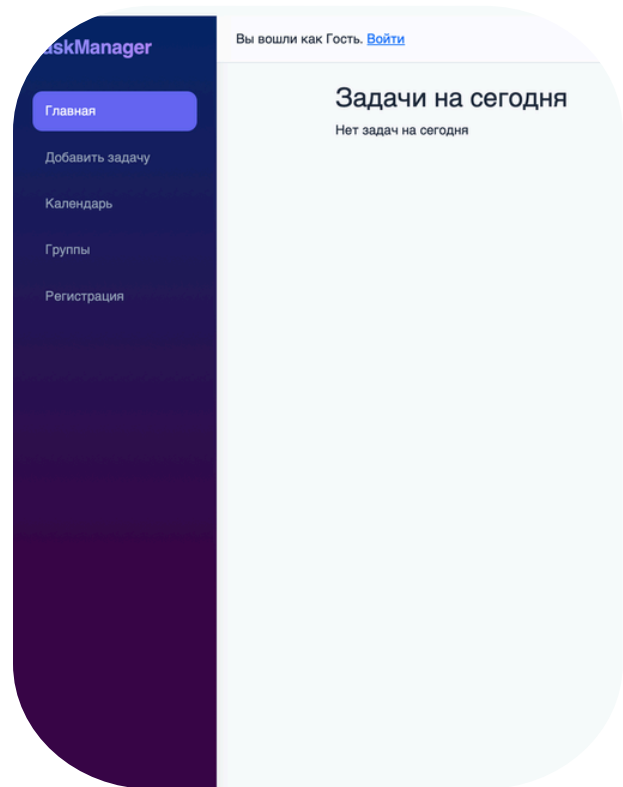
8

CONCLUSION



Project Overview

Modern web-based collaborative task management application.



- Personal tasks
- Group collaboration
- Comments & reactions
- Secure authentication

Core Features

1

CRUD task
management •
Group workspaces



2

Write
commentson
group task



3

Multi-group
support



Technical Stack

FRONTEND

Blazor Server



DATABASE

MSSQL Server



LANGUAGE

C#



BACKEND

ASP.NET Core



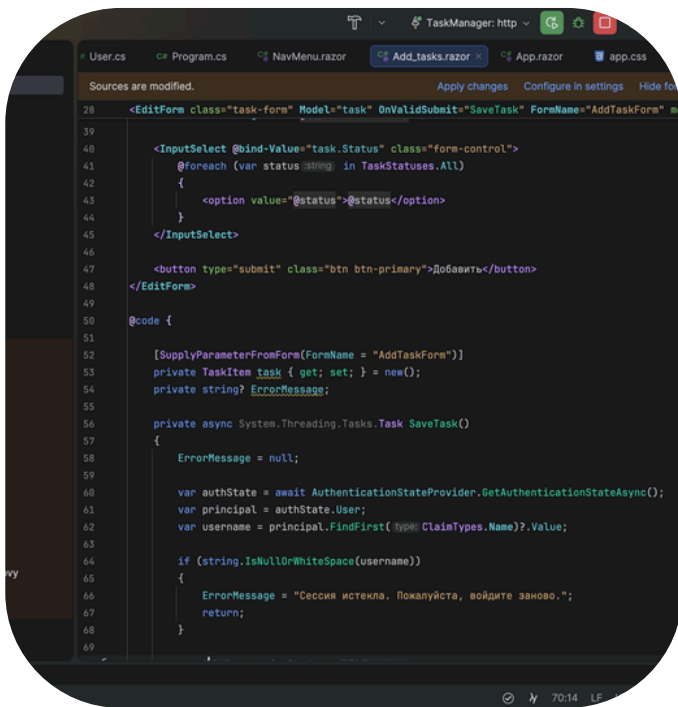
ORM

Entity Framework Core

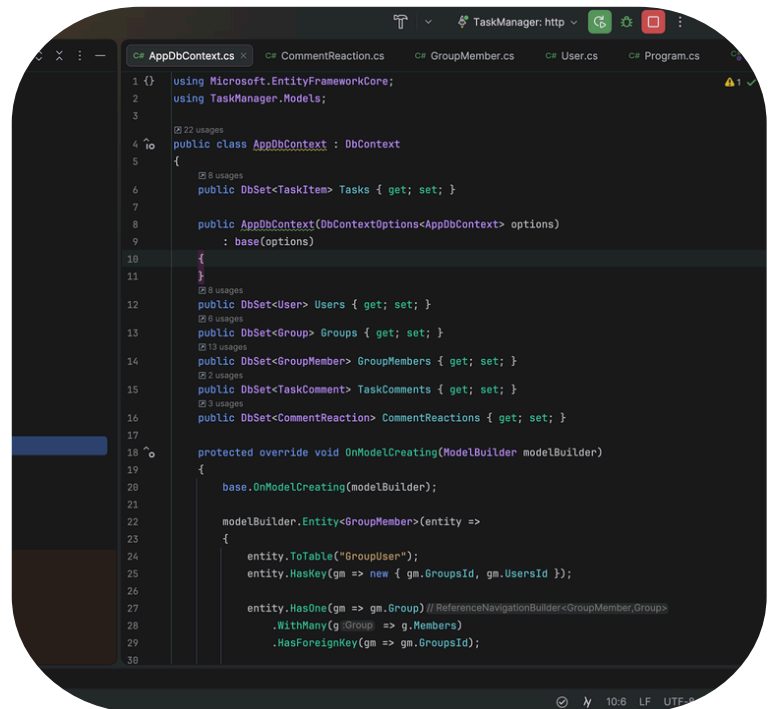


Architecture Overview

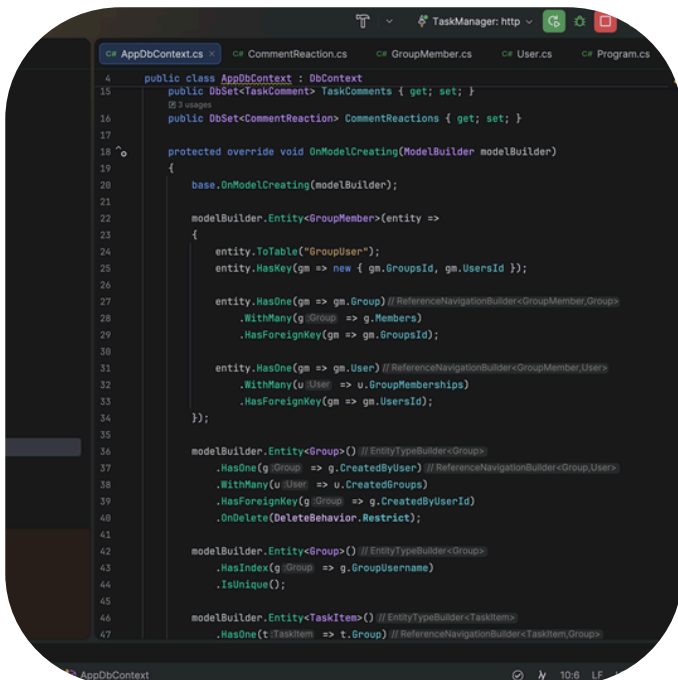
Blazor → ASP.NET → EF → MSSQL Server
UI Core Core Services



```
28 <EditForm class="task-form" Model="task" OnValidSubmit="SaveTask" FormName="AddTaskForm" ...
39
40 <InputSelect @bind-Value="task.Status" class="form-control">
41     @foreach (var status in TaskStatuses.All)
42     {
43         <option value="@status">@status</option>
44     }
45 </InputSelect>
46
47 <button type="submit" class="btn btn-primary">Добавить</button>
48 </EditForm>
49
50 @code {
51
52     [SupplyParameterFromForm(FormName = "AddTaskForm")]
53     private TaskItem task { get; set; } = new();
54     private string? ErrorMessage;
55
56     private async System.Threading.Tasks.Task SaveTask()
57     {
58         ErrorMessage = null;
59
60         var authState = await AuthenticationStateProvider.GetAuthenticationStateAsync();
61         var principal = authState.User;
62         var username = principal.FindFirst(ClaimTypes.Name)?.Value;
63
64         if (string.IsNullOrEmpty(username))
65         {
66             ErrorMessage = "Сессия истекла. Пожалуйста, войдите заново.";
67             return;
68         }
69     }
```



```
1 using Microsoft.EntityFrameworkCore;
2 using TaskManager.Models;
3
4 public class AppDbContext : DbContext
5 {
6     public DbSet<TaskItem> Tasks { get; set; }
7
8     public AppDbContext(DbContextOptions<AppDbContext> options)
9         : base(options)
10     {
11     }
12
13     public DbSet<User> Users { get; set; }
14     public DbSet<Group> Groups { get; set; }
15     public DbSet<GroupMember> GroupMembers { get; set; }
16     public DbSet<TaskComment> TaskComments { get; set; }
17     public DbSet<CommentReaction> CommentReactions { get; set; }
18
19     protected override void OnModelCreating(ModelBuilder modelBuilder)
20     {
21         base.OnModelCreating(modelBuilder);
22
23         modelBuilder.Entity<GroupMember>(entity =>
24         {
25             entity.ToTable("GroupUser");
26             entity.HasKey(gm => new { gm.GroupId, gm.UserId });
27
28             entity.HasOne(gm => gm.Group) // ReferenceNavigationBuilder<GroupMember, Group>
29                 .WithMany(g => g.Members)
30                 .HasForeignKey(gm => gm.GroupId);
31
32             entity.HasOne(gm => gm.User) // ReferenceNavigationBuilder<GroupMember, User>
33                 .WithMany(u => u.GroupMemberships)
34                 .HasForeignKey(gm => gm.UserId);
35         });
36
37         modelBuilder.Entity<Group>() // EntityTypeBuilder<Group>
38             .HasOne(g => g.CreatedByUser) // ReferenceNavigationBuilder<Group, User>
39             .WithMany(u => u.CreatedGroups)
40             .HasForeignKey(g => g.CreatedById)
41             .OnDelete(DeleteBehavior.Restrict);
42
43         modelBuilder.Entity<Group>() // EntityTypeBuilder<Group>
44             .HasIndex(g => g.GroupUsername)
45             .IsUnique();
46
47         modelBuilder.Entity<TaskItem>() // EntityTypeBuilder<TaskItem>
48             .HasOne(t => t.Group) // ReferenceNavigationBuilder<TaskItem, Group>
```



```
4 public class AppDbContext : DbContext
5 {
6     public DbSet<TaskComment> TaskComments { get; set; }
7     public DbSet<CommentReaction> CommentReactions { get; set; }
8
9     protected override void OnModelCreating(ModelBuilder modelBuilder)
10     {
11         base.OnModelCreating(modelBuilder);
12
13         modelBuilder.Entity<GroupMember>(entity =>
14         {
15             entity.ToTable("GroupUser");
16             entity.HasKey(gm => new { gm.GroupId, gm.UserId });
17
18             entity.HasOne(gm => gm.Group) // ReferenceNavigationBuilder<GroupMember, Group>
19                 .WithMany(g => g.Members)
20                 .HasForeignKey(gm => gm.GroupId);
21
22             entity.HasOne(gm => gm.User) // ReferenceNavigationBuilder<GroupMember, User>
23                 .WithMany(u => u.GroupMemberships)
24                 .HasForeignKey(gm => gm.UserId);
25         });
26
27         modelBuilder.Entity<Group>() // EntityTypeBuilder<Group>
28             .HasOne(g => g.CreatedByUser) // ReferenceNavigationBuilder<Group, User>
29             .WithMany(u => u.CreatedGroups)
30             .HasForeignKey(g => g.CreatedById)
31             .OnDelete(DeleteBehavior.Restrict);
32
33         modelBuilder.Entity<Group>() // EntityTypeBuilder<Group>
34             .HasIndex(g => g.GroupUsername)
35             .IsUnique();
36
37         modelBuilder.Entity<TaskItem>() // EntityTypeBuilder<TaskItem>
38             .HasOne(t => t.Group) // ReferenceNavigationBuilder<TaskItem, Group>
```

Data Models

AUTOMATIC MIGRATIONS AND SCHEMA SETUP USING EF CORE.

USER



GROUP



TASKCOMMENT



TASKITEM



TASKSTATUSES



Security Architecture

**PASSWOR
D HASHING**



**CSRF
PROTECTION**



**COOKIE
AUTHENTI
CATION**



**SECURE
SESSIONS**



API Endpoints

2

/AUTH/LOGIN

3

/AUTH/REGISTER

4

/LOGOUT

5

/TASKS

6

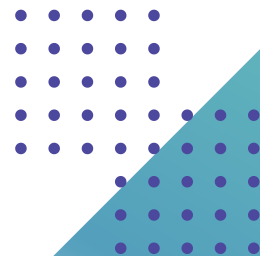
/GROUPS



Conclusion

The project demonstrates a complete web application with:

- Modern web framework (ASP.NET Core + Blazor)
- Secure authentication & authorization
- Complex database design with relationships
- Collaborative features for team productivity
- Practices in security and performance





ALI
KHANTSZIAN



Thank you!

